

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №3

Логирование с помощью zap. Ведение структурированных логов

Цель занятия

Освоить организацию структурированного логирования в backend-приложении на Go с использованием библиотеки zap для записи, анализа и сопровождения событий приложения.

Задачи занятия

1. Изучить назначение логирования в серверных приложениях.
2. Понять различие между обычными текстовыми логами и структурированными логами.
3. Освоить подключение и базовую настройку библиотеки zap.
4. Научиться записывать логи разных уровней: `debug`, `info`, `warn`, `error`.
5. Реализовать логирование HTTP-запросов и внутренних событий приложения.
6. Научиться добавлять в лог поля с контекстом: путь, метод, идентификатор запроса, время обработки, код ответа.
7. Отработать анализ логов при штатной работе и при возникновении ошибок.

Теоретическая часть

1. Зачем backend-приложению нужны логи

Логи — это один из основных инструментов наблюдаемости backend-системы. Именно по логам разработчик чаще всего понимает, что происходило в приложении: когда оно было запущено, какой запрос пришёл, какой маршрут был вызван, что вернул обработчик, где возникла ошибка, сколько времени заняла операция. В стандартной библиотеке Go пакет `log` описывается как простой `logging package`, который предоставляет тип `Logger` и стандартный логгер с выводом в `stderr`. ([Go Packages](#))

Если в учебном проекте логирование ещё можно воспринимать как вспомогательную деталь, то в реальном серверном приложении без логов практически невозможно качественно сопровождать систему. При возникновении ошибки разработчик почти всегда начинает анализ именно с журналов событий.

2. Обычные и структурированные логи

Самый простой вариант — писать строковые сообщения, например:

```
user      15      requested    /orders    at      10:31.
```

Такой подход понятен человеку, но его неудобно обрабатывать автоматически. Сложнее искать по полям, фильтровать по пользователю, строить агрегации и анализировать большой поток записей.

Структурированное логирование строится иначе: запись лога содержит сообщение, уровень важности и набор полей в формате ключ-значение. В документации `log/slog` это прямо описано как

structured logging, где записи содержат message, severity level и дополнительные attributes в виде key-value pairs. Аналогично zap позиционируется как fast, structured, leveled logging, а logrus — как structured logger for Go. ([Go Packages](#))

Именно поэтому структурированные логи удобнее для серверов, API и микросервисов: их проще фильтровать, передавать в системы наблюдаемости и анализировать при диагностике.

3. Почему в этой работе используется zap

В этой практической работе основным инструментом будет библиотека zap. В её официальной документации отдельно указано, что пакет предоставляет fast, structured, leveled logging, а также различает Logger и SugaredLogger. Для сценариев, где важна простота использования, но не критичен каждый микросекундный оверхед, документация рекомендует SugaredLogger; для более чувствительных к производительности случаев используется Logger. ([Go Packages](#))

Для учебной практики zap удобен тем, что позволяет быстро получить структурированные JSON-логи и одновременно познакомиться с подходом, который часто используется в промышленной backend-разработке.

4. Где в приложении нужно логировать события

Логирование в серверном приложении не должно быть хаотичным. Обычно логируются:

- запуск и остановка сервиса;
- ошибки конфигурации;
- входящие HTTP-запросы;

время обработки запроса;
ошибки валидации и внутренние ошибки;
важные бизнес-события.

Если логировать всё подряд без системы, то журналы быстро превращаются в шум. Если, наоборот, логировать слишком мало, то при диагностике не хватит информации.

5. Уровни логирования

В практике backend-разработки используются разные уровни логов. В `zap` логирование является уровневым, то есть записи могут иметь разные `severity levels`. В нашей работе будут использоваться такие уровни:

`Debug` — подробные технические сведения для отладки;

`Info` — нормальные рабочие события;

`Warn` — ситуация отклоняется от ожидаемой, но приложение ещё продолжает работать;

`Error` — операция завершилась ошибкой. ([Go Packages](#))

Разделение по уровням важно, потому что в `production`-среде обычно не хотят постоянно видеть весь поток отладочной информации, но хотят быстро находить предупреждения и ошибки.

6. Что мы сделаем в этой работе

В данной практике будет создан небольшой HTTP-сервис на Go с маршрутом проверки состояния и маршрутом получения информации о студенте.

В приложение будет встроен `zap`-логгер.

Будут реализованы:

- логирование старта сервиса;

- логирование входящих HTTP-запросов через middleware;
- логирование успешной обработки запроса;
- логирование ошибок, например неверного идентификатора или отсутствующего объекта;
- структурированные поля: HTTP-метод, путь, код ответа, длительность обработки, идентификатор студента.

В результате студент увидит, как именно structured logging помогает сопровождать backend-сервис.

Практическая часть

Общая идея проекта

В этой работе создаётся учебное HTTP-приложение с двумя маршрутами:

GET /health — проверка работоспособности сервиса;

GET /students/{id} — получение информации о студенте по идентификатору.

Приложение будет использовать zap для записи логов в консоль в структурированном формате.

Рекомендуемая структура проекта

```
pz3-logging/  
  cmd/  
    server/  
      main.go  
  internal/  
    httpapi/  
      handler.go  
      middleware.go  
      response_writer.go  
    student/  
      model.go  
      repo.go  
  pkg/  
    logger/  
      logger.go  
  go.mod
```

Шаг 1. Создание проекта

Откройте PowerShell и создайте папку проекта:

```
mkdir pz3-logging  
cd pz3-logging  
go mod init example.com/pz3-logging
```

```
mkdir cmd
mkdir cmd\server
mkdir internal
mkdir internal\httpapi
mkdir internal\student
mkdir pkg
mkdir pkg\logger
```

Для macOS/Linux:

```
mkdir -p pz3-logging/cmd/server
mkdir -p pz3-logging/internal/httpapi
mkdir -p pz3-logging/internal/student
mkdir -p pz3-logging/pkg/logger
cd pz3-logging
go mod init example.com/pz3-logging
```

Шаг 2. Подключение библиотеки zap

Установите зависимость:

```
go get go.uber.org/zap
```

Официальная документация пакета `go.uber.org/zap` описывает библиотеку как fast, structured, leveled logging for Go. ([Go Packages](#))

Шаг 3. Модель студента

Создайте файл `internal/student/model.go`:

```
package student

type Student struct {
    ID          int64  `json:"id"`
    FullName    string `json:"full_name"`
    Group       string `json:"group"`
    Email       string `json:"email"`
}
```


Шаг 4. Репозиторий с тестовыми данными

Создайте файл `internal/student/repo.go`:

```
package student

import "errors"

var ErrStudentNotFound = errors.New("student not found")

type Repo struct {
    data map[int64]Student
}

func NewRepo() *Repo {
    return &Repo{
        data: map[int64]Student{
            1: {
                ID:          1,
                FullName:    "Иванов Иван Иванович",
                Group:       "ИББО-01-25",
                Email:       "ivanov@example.com",
            },
            2: {
                ID:          2,
                FullName:    "Петрова Мария Сергеевна",
                Group:       "ИББО-02-25",
                Email:       "petrova@example.com",
            },
            3: {
                ID:          3,
                FullName:    "Сидоров Алексей Андреевич",
                Group:       "ИББО-03-25",
                Email:       "sidorov@example.com",
            },
        },
    }
}
```

```
func (r *Repo) GetByID(id int64) (Student, error) {
    st, ok := r.data[id]
    if !ok {
        return Student{}, ErrStudentNotFound
    }
    return st, nil
}
```

Шаг 5. Настройка логгера

Создайте файл `pkg/logger/logger.go`:

```
package logger

import "go.uber.org/zap"

func New() (*zap.Logger, error) {
    cfg := zap.NewProductionConfig()
    cfg.OutputPaths = []string{"stdout"}
    cfg.ErrorOutputPaths = []string{"stderr"}

    return cfg.Build()
}
```

Пояснение

В `zap` есть разные варианты конфигурации. Для `production`-сценариев типично используют `production`-конфигурацию, которая ориентирована на структурированный вывод. Документация `zap` описывает библиотеку как `structured logger` и даёт разные варианты выбора и настройки логгера. ([Go Packages](#))

Шаг 6. Обёртка над `ResponseWriter` для фиксации статуса

Чтобы корректно логировать HTTP-статус ответа, создайте файл `internal/httpapi/response_writer.go`:

```
package httpapi
```

```

import "net/http"

type LoggingResponseWriter struct {
    http.ResponseWriter
    statusCode int
}

func NewLoggingResponseWriter(w http.ResponseWriter)
*LoggingResponseWriter {
    return &LoggingResponseWriter{
        ResponseWriter: w,
        statusCode:     http.StatusOK,
    }
}

func (lrw *LoggingResponseWriter) WriteHeader(code int) {
    lrw.statusCode = code
    lrw.ResponseWriter.WriteHeader(code)
}

func (lrw *LoggingResponseWriter) StatusCode() int {
    return lrw.statusCode
}

```

Шаг 7. Middleware логирования запросов

Создайте файл `internal/httpapi/middleware.go`:

```

package httpapi

import (
    "net/http"
    "time"

    "go.uber.org/zap"
)

func LoggingMiddleware(log *zap.Logger, next http.Handler)
http.Handler {

```

```

        return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
            start := time.Now()

            lrw := NewLoggingResponseWriter(w)
            requestID := time.Now().UnixNano()

            log.Info("incoming request",
                zap.Int64("request_id", requestID),
                zap.String("method", r.Method),
                zap.String("path", r.URL.Path),
                zap.String("remote_addr", r.RemoteAddr),
            )

            next.ServeHTTP(lrw, r)

            duration := time.Since(start)

            log.Info("request completed",
                zap.Int64("request_id", requestID),
                zap.String("method", r.Method),
                zap.String("path", r.URL.Path),
                zap.Int("status_code", lrw.StatusCode()),
                zap.Duration("duration", duration),
            )
        })
    }
}

```

Пояснение

Структурированный лог удобен тем, что в него можно добавить отдельные поля, а не склеивать всё в одну строку. Именно такой подход является базовым для structured logging: сообщение плюс набор key-value полей. Это соответствует модели, описанной и для zap, и для slog. ([Go Packages](#))

Шаг 8. HTTP-обработчики

Создайте файл `internal/httpapi/handler.go`:

```
package httpapi

import (
    "encoding/json"
    "net/http"
    "strconv"
    "strings"

    "example.com/pz3-logging/internal/student"
    "go.uber.org/zap"
)

type Handler struct {
    repo *student.Repo
    log  *zap.Logger
}

func NewHandler(repo *student.Repo, log *zap.Logger) *Handler {
    return &Handler{
        repo: repo,
        log:  log,
    }
}

func (h *Handler) Health(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodGet {
        h.log.Warn("method not allowed for health endpoint",
            zap.String("method", r.Method),
            zap.String("path", r.URL.Path),
        )
        http.Error(w, "method not allowed",
            http.StatusMethodNotAllowed)
        return
    }

    h.log.Debug("health endpoint called")
}
```

```

        w.Header().Set("Content-Type", "application/json; charset=utf-8")

        _ = json.NewEncoder(w).Encode(map[string]string{
            "status": "ok",
        })
    }

    func (h *Handler) GetStudentByID(w http.ResponseWriter, r
    *http.Request) {
        if r.Method != http.MethodGet {
            h.log.Warn("method not allowed for student endpoint",
                zap.String("method", r.Method),
                zap.String("path", r.URL.Path),
            )
            http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
            return
        }

        path := strings.TrimPrefix(r.URL.Path, "/students/")
        if path == "" || path == r.URL.Path {
            h.log.Warn("student id is missing",
                zap.String("path", r.URL.Path),
            )
            http.Error(w, "student id is required",
http.StatusBadRequest)
            return
        }

        id, err := strconv.ParseInt(path, 10, 64)
        if err != nil {
            h.log.Warn("invalid student id",
                zap.String("raw_id", path),
                zap.Error(err),
            )
            http.Error(w, "invalid student id",
http.StatusBadRequest)

```

```

        return
    }

    st, err := h.repo.GetByID(id)
    if err != nil {
        h.log.Error("student not found",
            zap.Int64("student_id", id),
            zap.Error(err),
        )
        http.Error(w, "student not found", http.StatusNotFound)
        return
    }

    h.log.Info("student returned successfully",
        zap.Int64("student_id", st.ID),
        zap.String("group", st.Group),
    )

    w.Header().Set("Content-Type", "application/json; charset=utf-8")

    _ = json.NewEncoder(w).Encode(st)
}

```

Шаг 9. Точка входа приложения

Создайте файл `cmd/server/main.go`:

```

package main

import (
    "log"
    "net/http"

    "example.com/pz3-logging/internal/httpapi"
    "example.com/pz3-logging/internal/student"
    applogger "example.com/pz3-logging/pkg/logger"
    "go.uber.org/zap"
)

```

```

func main() {
    logger, err := applogger.New()
    if err != nil {
        log.Fatal(err)
    }
    defer logger.Sync()

    repo := student.NewRepo()
    handler := httpapi.NewHandler(repo, logger)

    mux := http.NewServeMux()
    mux.HandleFunc("/health", handler.Health)
    mux.HandleFunc("/students/", handler.GetStudentByID)

    rootHandler := httpapi.LoggingMiddleware(logger, mux)

    logger.Info("server is starting",
        zap.String("addr", ":8080"),
    )

    if err := http.ListenAndServe(":8080", rootHandler); err != nil
{
        logger.Fatal("server failed",
            zap.Error(err),
        )
    }
}

```

Шаг 10. Запуск приложения

В корне проекта выполните:

```
go run ./cmd/server
```

Ожидается, что приложение запустится на порту 8080.

В консоли появится структурированная запись о старте сервера.

Шаг 11. Проверка маршрута health

Откройте второе окно терминала и выполните:

```
curl http://localhost:8080/health
```

Ожидаемый ответ:

```
{"status": "ok"}
```

При этом в консоли сервера должны появиться логи о входящем запросе и о завершении обработки.

Пример структуры записи лога может быть таким:

```
{"level": "info", "msg": "incoming  
request", "method": "GET", "path": "/health"}
```

Именно такой формат и является главным результатом практической работы: лог несёт не только сообщение, но и отдельные поля, которые можно анализировать по ключам. Это соответствует общей модели structured logging, описанной в официальных материалах zap и slog. ([Go Packages](#))

Шаг 12. Проверка маршрута получения студента

Выполните:

```
curl http://localhost:8080/students/1
```

Ожидаемый ответ:

```
{  
  "id": 1,  
  "full_name": "Иванов Иван Иванович",  
  "group": "ИВБО-01-25",  
  "email": "ivanov@example.com"  
}
```

В логах должны появиться записи:

- о входящем запросе;

- об успешной выдаче студента;
- о завершении обработки запроса.

Шаг 13. Проверка ошибки в идентификаторе

Выполните:

```
curl http://localhost:8080/students/abc
```

Ожидаемый результат — ошибка 400 Bad Request.

В логах должна быть запись уровня warn с информацией о том, что идентификатор передан в неверном формате.

Шаг 14. Проверка отсутствующего студента

Выполните:

```
curl http://localhost:8080/students/999
```

Ожидаемый результат — ошибка 404 Not Found.

В логах должна быть запись уровня error с полем student_id.

Шаг 15. Что получилось в результате

В ходе практической работы студент реализовал backend-приложение, в котором:

- используется структурированный логгер zap;
- логируются входящие HTTP-запросы;
- фиксируются код ответа и длительность обработки;
- разделяются уровни логов;
- ошибки и рабочие события сопровождаются

дополнительными полями.

Это уже не просто вывод сообщений в консоль, а полноценная база для дальнейшей наблюдаемости и диагностики сервиса.

Что важно понять по итогам работы

Обычный текстовый лог полезен только до определённого масштаба. Как только в приложении появляется много маршрутов, пользователей, ошибок и сервисных операций, строкового логирования становится недостаточно.

Структурированные логи решают эту проблему, потому что каждое событие представлено в виде записи с полями. Это значит, что можно искать не только по тексту сообщения, но и, например, по `student_id`, `status_code`, `path` или `duration`.

Для backend-приложений это особенно важно, потому что сопровождение сервиса почти всегда строится вокруг ответов на вопросы:

- какой запрос пришёл;
- что произошло внутри;
- какой ответ был возвращён;
- сколько это заняло времени;
- где именно возникла ошибка.

Кратко об альтернативе: logrus

Вместо `zap` можно использовать `logrus`. Его официальный репозиторий описывает его как `structured, pluggable logger for Go` и подчёркивает совместимость API со стандартной библиотекой. При этом в том же репозитории прямо указано, что `logrus` находится в `maintenance mode`, то есть новые возможности активно не развиваются.

Поэтому для учебной и промышленной практики сегодня чаще выбирают zap или slog, хотя logrus по-прежнему встречается в существующих проектах. ([GitHub](#))

Дополнительные задания

Вариант 1. Добавить логирование в файл

Измените конфигурацию логгера так, чтобы логи писались не только в stdout, но и в файл.

Вариант 2. Добавить поле request_id в ответ

Передавайте идентификатор запроса не только в лог, но и в HTTP-заголовок ответа.

Вариант 3. Добавить debug-логирование бизнес-операции

Перед чтением студента из репозитория добавьте debug-запись о начале поиска.

Вариант 4. Реализовать логирование POST-запроса

Добавьте маршрут создания студента и журналируйте тело запроса, ошибки валидации и успешное создание записи.

Контрольные вопросы

1. Зачем backend-приложению нужно логирование?
2. Чем обычный текстовый лог отличается от структурированного?
3. Что означает structured logging?
4. Какие уровни логирования используются в этой работе?
5. Почему полезно логировать HTTP-метод, путь и статус ответа?
6. Зачем в лог добавляют время выполнения запроса?

7. Почему логирование ошибок должно содержать дополнительный контекст?
8. В чём практическое преимущество zap?
9. Что означает maintenance mode у logrus? ([GitHub](#))
10. Почему structured logging особенно важен для микросервисов и backend API?

Типичные ошибки

Если приложение запускается, но логи не видны, сначала проверьте, не перенаправлен ли вывод и корректно ли настроены `OutputPaths`.

Если HTTP-статус в логах всегда 200, значит код ответа не перехватывается через собственную обёртку над `ResponseWriter`.

Если уровни логирования не различаются, проверьте, какие методы вызываются: `Info`, `Warn`, `Error`, `Debug`.

Если в логах нет полезного контекста, значит сообщения записываются как обычный текст без полей — это сводит преимущества structured logging почти к нулю.

Если при завершении приложения появляется ошибка `Sync`, в локальной среде это может быть связано со спецификой `stdout/stderr`, поэтому такой случай обычно не считается критической проблемой учебного примера.